OperatorNotAllowedInGraphError: iterating over `tf.Tensor` is not allowed:

https://github.com/tensorflow/quantum/issues/713

ROW 133

OperatorNotAllowedInGraphError: iterating over `tf.Tensor` is not allowed: AutoGraph did not convert this function. This might indicate you are trying to use an unsupported feature. #713

New issue

Closed

Shuhul24 opened on Aug 19, 2022 · edited by Shuhul24 — Edits

I wrote a function as below:

```
def convert_to_circuit(image):
  # values = tf.reshape(image, [-1])
  images = image.numpy()
  values = np.ndarray.flatten(images)
  qubits = cirq.GridQubit.rect(4, 4)
  circuit = cirq.Circuit()
  for i in range(4):
    for j in range(4):
      k = (4 * i) + j
      circuit.append(cirq.H(cirq.GridQubit(i, j)))
      circuit.append(cirq.Rx(rads=values[k]).on(cirq.GridQubit(i, j)))
  return circuit
```

While running it for `tensorflow_quantum.layers.PQC`, I need to convert a batch of images into these circuit and save them in a list.
After running it, using @tf.function decorator, I am getting an OperatorNotAllowedInGraphError.

```
@tf.function
def train_step(images):
    noise = tf.random.normal([BATCH_SIZE, noise_dim, 1])

    with tf.GradientTape() as gen_tape, tf.GradientTape() as disc_tape:
      generated_images = generator(noise, training=True)

      real_in = [convert_to_circuit(x) for x in images]

      real_input = tfq.convert_to_tensor(real_in)

      fake_in = [convert_to_circuit(x) for x in generated_images]
      fake_input = tfq.convert_to_tensor(fake_in)

      real_output = discriminator(real_input, training=True)

      fake_output = discriminator(fake_input, training=True)

      gen_loss = generator_loss(fake_output)
      disc_loss = discriminator_loss(real_output, fake_output)

    gradients_of_generator = gen_tape.gradient(gen_loss, generator.trainable_variables)
    gradients_of_discriminator = disc_tape.gradient(disc_loss, discriminator.trainable_variables)

    generator_optimizer.apply_gradients(zip(gradients_of_generator, generator.trainable_variables))
    discriminator_optimizer.apply_gradients(zip(gradients_of_discriminator, discriminator.trainable_variables))
```

The error shown as below:

```
---------------------------------------------------------------------------
OperatorNotAllowedInGraphError            Traceback (most recent call last)
[<ipython-input-77-d152560ca122>](https://localhost:8080/#) in <module>
----> 1 train(train_dataset, EPOCHS)

2 frames
[/usr/local/lib/python3.7/dist-packages/tensorflow/python/framework/func_graph.py](https://localhost:8080/#) i
   1145           except Exception as e:  # pylint:disable=broad-except
   1146             if hasattr(e, "ag_error_metadata"):
-> 1147               raise e.ag_error_metadata.to_exception(e)
   1148             else:
   1149               raise

OperatorNotAllowedInGraphError: in user code:

    File "<ipython-input-75-acefef8060ba>", line 12, in train_step  *
        real_in = tf.numpy_function(convert_to_circuit, images, tf.float32)
```

Assignees
No one assigned

Labels
No labels

Type
No type

Projects
No projects

Milestone
No milestone

Relationships
None yet

Development
No branches or pull requests

Notifications Customize

Subscribe

You're not receiving notifications from this thread.

Participants

```
OperatorNotAllowedInGraphError: iterating over `tf.Tensor` is not allowed: AutoGraph did convert this func
```

The error also comes with a comment that "This might indicate you are trying to use an unsupported feature".

Other information:

```
tensorflow-quantum==0.7.2
tensorflow==2.8.2
cirq-core==0.13.1
cirq-google==0.13.1
```

lockwo on Aug 19, 2022 — Contributor

That a general rule of tensorflow: it can't construct a graph from iterating over tensors (which is the unsupported feature). A trivial example is:

```
@tf.function
def test(x):
  return [i + 1 for i in x]

t = tf.convert_to_tensor([1, 2, 3])
test(t)
```

The way around this is to make autograph compatible functions in place of the illegal operations. I.e. convert list comprehensions to tf native (or compatible) functions. In the example I provided this would be as easy as

```
@tf.function
def test(x):
  return x + 1

t = tf.convert_to_tensor([1, 2, 3])
test(t)
```

In your example, there may be a way to convert that to an autograph-able function (although in my experience, cirq and autograph don't always play nice), but what I recommend doing is simply extracting the gradient code to a new function and only using the tf.function decorator on that (since converting images to circuits is pretty trivial and probably isn't the bottleneck of the code). E.g.

```
@tf.function
def grad(real_input, fake_input):
    with tf.GradientTape() as gen_tape, tf.GradientTape() as disc_tape:
      real_output = discriminator(real_input, training=True)
      fake_output = discriminator(fake_input, training=True)

      gen_loss = generator_loss(fake_output)
      disc_loss = discriminator_loss(real_output, fake_output)

    gradients_of_generator = gen_tape.gradient(gen_loss, generator.trainable_variables)
    gradients_of_discriminator = disc_tape.gradient(disc_loss, discriminator.trainable_variables)

    generator_optimizer.apply_gradients(zip(gradients_of_generator, generator.trainable_variables))
    discriminator_optimizer.apply_gradients(zip(gradients_of_discriminator, discriminator.trainable_variables)

def train_step(images):
    noise = tf.random.normal((BATCH_SIZE, noise_dim, 1))

    generated_images = generator(noise, training=True)

    real_in = [convert_to_circuit(x) for x in images]

    real_input = tfq.convert_to_tensor(real_in)

    fake_in = [convert_to_circuit(x) for x in generated_images]
    fake_input = tfq.convert_to_tensor(fake_in)
    grad(real_input, fake_input)
```

Shuhul24 on Aug 20, 2022 · edited by Shuhul24 — Edits — Author

This leads into a new error as below:

```
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
[<ipython-input-98-d152560ca122>](https://localhost:8080/#) in <module>
----> 1 train(train_dataset, EPOCHS)

3 frames
[/usr/local/lib/python3.7/dist-packages/tensorflow/python/framework/func_graph.py](https://localhost:8080/#) i
   1145           except Exception as e:  # pylint:disable=broad-except
   1146             if hasattr(e, "ag_error_metadata"):
-> 1147               raise e.ag_error_metadata.to_exception(e)
   1148             else:
   1149               raise

ValueError: in user code:

    File "<ipython-input-78-92fb3499de18>", line 15, in grad  *
        generator_optimizer.apply_gradients(zip(gradients_of_generator, generator.trainable_variables))
    File "/usr/local/lib/python3.7/dist-packages/keras/optimizer_v2/optimizer_v2.py", line 633, in apply_gradi
        grads_and_vars = optimizer_utils.filter_empty_gradients(grads_and_vars)
    File "/usr/local/lib/python3.7/dist-packages/keras/optimizer_v2/utils.py", line 73, in filter_empty_gradie
        raise ValueError(f"No gradients provided for any variable: {variable}. "

    ValueError: No gradients provided for any variable: (['dense/kernel:0', 'dense/bias:0', 'conv2d/kernel:0',
```

What leads to this new error?

lockwo on Aug 20, 2022 — Contributor

The generator is not called within the gradient tape (so it can't find gradients for it). I didn't realize the generator was called before the conversion, which means my initial proposal may not be feasible. You will probably have to write the conversion into autographable code. There are probably several ways to do this. I might do something like: parameterize the encoder circuit with symbols, then resolve the parameters with the images. Not sure if you could do the exact code below, but this is the direction I would go in (this code runs for me, so it should be auto-graphable):

```
def convert_to_circuit(param):
  qubits = cirq.GridQubit.rect(4, 4)
  circuit = cirq.Circuit()
  for i in range(4):
    for j in range(4):
      k = (4 * i) + j
      circuit.append(cirq.H(cirq.GridQubit(i, j)))
      circuit.append(cirq.Rx(rads=param[k]).on(cirq.GridQubit(i, j)))
  return circuit

@tf.function
def test(param, tensor_c):
  images = tf.random.uniform(shape=[10, 16]) # [batch, flattened_size]
  tensor_c = tf.tile(tensor_c, [10])
  encoded_images = tfq.resolve_parameters(tensor_c, [s.name for s in param], images)
  return encoded_images

params = sympy.symbols('q0:16')
circuit = tfq.convert_to_tensor([convert_to_circuit(params)])
res = test(params, circuit)
```

Shuhul24 on Aug 20, 2022 — Author

Actually, your previous code snippet ran for me! But the error that got raised was because of the fact that `generator_optimizer.apply_gradients(zip(gradients_of_generator, generator.trainable_variables))` was not running successfully because `gradient_of_generator` was resulting in a `None` value. gist can be shown here. Other issues related to it. here. It's more related to `tensorflow` issue. Can you please help me figure this out?

successfully because `gradient_of_generator` was resulting in a `None` value. gist can be shown here. Other issues related to it. here. It's more related to `tensorflow` issue. Can you please help me figure this out?

lockwo on Aug 20, 2022 Contributor ···

It's like I said above, "The generator is not called within the gradient tape (so it can't find gradients for it)". You have to call `generator(noise, training=True)` inside gradient tape if you want gradients for it.

lockwo on Aug 20, 2022 · edited by lockwo Edits Contributor ···

For example:

Won't work:

```
gen = tf.keras.models.Sequential([
  tf.keras.layers.Flatten(input_shape=(28, 28)),
  tf.keras.layers.Dense(128, activation='relu'),
  tf.keras.layers.Dropout(0.2),
  tf.keras.layers.Dense(10)
])

dis = tf.keras.models.Sequential([
  tf.keras.layers.Flatten(input_shape=(28, 28)),
  tf.keras.layers.Dense(128, activation='relu'),
  tf.keras.layers.Dropout(0.2),
  tf.keras.layers.Dense(10)
])

dis_opt = tf.keras.optimizers.Adam()
gen_opt = tf.keras.optimizers.Adam()

def grad():
  inputs = tf.random.uniform(shape=[10, 28, 28])
  x = gen(inputs, training=True)
  with tf.GradientTape() as tape1, tf.GradientTape() as tape2:

    y = dis(inputs, training=True)

  grads = tape1.gradient(x, gen.trainable_variables)
  gen_opt.apply_gradients(zip(grads, gen.trainable_variables))
  grads = tape2.gradient(y, dis.trainable_variables)
  dis_opt.apply_gradients(zip(grads, dis.trainable_variables))

grad()
```

Will work:

```
gen = tf.keras.models.Sequential([
  tf.keras.layers.Flatten(input_shape=(28, 28)),
  tf.keras.layers.Dense(128, activation='relu'),
  tf.keras.layers.Dropout(0.2),
  tf.keras.layers.Dense(10)
])

dis = tf.keras.models.Sequential([
  tf.keras.layers.Flatten(input_shape=(28, 28)),
  tf.keras.layers.Dense(128, activation='relu'),
  tf.keras.layers.Dropout(0.2),
  tf.keras.layers.Dense(10)
])

dis_opt = tf.keras.optimizers.Adam()
gen_opt = tf.keras.optimizers.Adam()

def grad():
  inputs = tf.random.uniform(shape=[10, 28, 28])

  with tf.GradientTape() as tape1, tf.GradientTape() as tape2:
    x = gen(inputs, training=True)
    y = dis(inputs, training=True)
```

```
    x = gen(inputs, training=True)
    y = dis(inputs, training=True)

  grads = tape1.gradient(x, gen.trainable_variables)
  gen_opt.apply_gradients(zip(grads, gen.trainable_variables))
  grads = tape2.gradient(y, dis.trainable_variables)
  dis_opt.apply_gradients(zip(grads, dis.trainable_variables))

grad()
```

Shuhul24 on Aug 20, 2022 · edited by Shuhul24 — Edits — Author

But I have called the generator in the `GradientTape()`
The code snippet is below:

```
    with tf.GradientTape() as gen_tape, tf.GradientTape() as disc_tape:
      generated_images = generator(noise, training=True)
      gen_tape.watch(generator.trainable_weights)


      fake_in = [convert_to_circuit(x) for x in generated_images]
      fake_input = tfq.convert_to_tensor(fake_in)

      real_output = discriminator(real_input, training=True)
      fake_output = discriminator(fake_input, training=True)


      gen_loss = generator_loss(fake_output)
      disc_loss = discriminator_loss(real_output, fake_output)
```

lockwo on Aug 20, 2022 · edited by lockwo — Edits — Contributor

Because `convert_to_circuit(x)` is not a differentiable function. As a simple proof of concept (this code will fail because it can't find gradients): (also even if it was differentiable, it probably wouldn't be autographable (because it is list comprehension)).

```
def grad():
  inputs = tf.random.uniform(shape=[1, 28, 28])
  with tf.GradientTape() as gen_tape, tf.GradientTape() as disc_tape:
    generated_images = gen(inputs, training=True)
    gen_tape.watch(gen.trainable_weights)

    fake_in = [convert_to_circuit(x) for x in generated_images]
    fake_input = tfq.convert_to_tensor(fake_in)

    gen_loss = tfq.layers.Expectation()(fake_input, symbol_names=[], symbol_values=[[]], operators=[cirq.Z(ci

  grads = gen_tape.gradient(gen_loss, gen.trainable_variables)
  gen_opt.apply_gradients(zip(grads, gen.trainable_variables))


grad()
```

Shuhul24 on Aug 21, 2022 · edited by Shuhul24 — Edits — Author

Okay. I think the error is in the function `convert_to_circuit` or `tfq.convert_to_tensor` (or both). So what can be done in this case? Should I convert the function into autograph-able using `tf.autograph.to_graph()` ?
Also, how to check whether the function is differentiable?

Shuhul24 on Aug 21, 2022 · edited by Shuhul24 — Edits — Author

Okay. I think the error is in the function `convert_to_circuit` or `tfq.convert_to_tensor` (or both). So what can be done in this case? Should I convert the function into autograph-able using `tf.autograph.to_graph()` ?
Also, how to check whether the function is differentiable?

lockwo on Aug 21, 2022 · edited by lockwo — Edits — Contributor

Convert to tensor should be differentiable. I'm not sure if there is a general method to check if something is differentiable (I usually just run the op through gradient tape and test). To make convert to circuit differentiable, one way would be to use ControlledPQC and create the circuit before hand and use the generated weights as inputs to the ControlledPQC. There's probably other ways, but that's my go to for custom circuit inputs.

Shuhul24 on Aug 23, 2022 · edited by Shuhul24 — Edits — Author

I think `tfq.layers.ControlledPQC` must work, but I am having trouble with building the `tf.keras.Sequential` model for it, as I built with `tfq.layers.PQC` . Can you give me a sample code for the same (for building a `Sequential` model with `ControlledPQC` in it) ?

lockwo on Aug 23, 2022 · edited by lockwo — Edits — Contributor

As with most things in TFQ, it's probably easiest to use a custom layer (a valuable lesson I learned from @/MichaelBroughton). I made up a discriminator circuit and generator, but the workflow should look something like the code below. This is able to encode the output of the generator using a ControlledPQC (in which the weights of the encoder circuit are controlled by said output). This is structurally combined with the discriminator, whose weights are learned (and managed) by TFQ inside the custom layer. This code produces gradients for me, and is wrappable in a `tf.function` decorator for speed. Hopefully this is a step in direction of what you are looking for. The losses are copied from: https://www.tensorflow.org/tutorials/generative/dcgan and the other code is mostly Frankenstein-ed from:
https://github.com/lockwo/quantum_computation/blob/master/TFQ/RL_QVC/atari_qddqn.py and
https://github.com/lockwo/quantum_computation/blob/master/TFQ/RL_QVC/policies.py

```
import tensorflow as tf
import tensorflow_quantum as tfq
import cirq
import sympy
import numpy as np

cross_entropy = tf.keras.losses.BinaryCrossentropy(from_logits=True)

def discriminator_loss(real_output, fake_output):
    real_loss = cross_entropy(tf.ones_like(real_output), real_output)
    fake_loss = cross_entropy(tf.zeros_like(fake_output), fake_output)
    total_loss = real_loss + fake_loss
    return total_loss

def generator_loss(fake_output):
    return cross_entropy(tf.ones_like(fake_output), fake_output)

def convert_to_circuit(params):
  qubits = cirq.GridQubit.rect(4, 4)
  circuit = cirq.Circuit()
  for i in range(4):
    for j in range(4):
      k = (4 * i) + j
      circuit.append(cirq.H(cirq.GridQubit(i, j)))
      circuit.append(cirq.Rx(rads=params[k]).on(cirq.GridQubit(i, j)))
  return circuit

def u_ent(qubits, ps):
    c = cirq.Circuit()
    for i in range(len(qubits)):
```

```
            c += cirq.rz(ps[i]).on(qubits[i])
        for i in range(len(qubits)):
            c += cirq.ry(ps[i + len(qubits)]).on(qubits[i])
        for i in range(len(qubits) - 1):
            c += cirq.CZ(qubits[i], qubits[i+1])
        c += cirq.CZ(qubits[-1], qubits[0])
        return c

def convert_to_circuit(params):
  qubits = cirq.GridQubit.rect(4, 4)
  circuit = cirq.Circuit()
  for i in range(4):
    for j in range(4):
      k = (4 * i) + j
      circuit.append(cirq.H(cirq.GridQubit(i, j)))
      circuit.append(cirq.Rx(rads=params[k]).on(cirq.GridQubit(i, j)))
  return circuit

class Hybrid_Dis(tf.keras.layers.Layer):
    def __init__(self) -> None:
        super().__init__()
        ops = [cirq.Z(cirq.GridQubit(0, 0))]
        convert_params = sympy.symbols("q0:16")
        qubits = cirq.GridQubit.rect(4, 4)
        ent_params = sympy.symbols("e0:32")
        convert_circuit = convert_to_circuit(convert_params)
        dis_circuit = u_ent(qubits, ent_params)
        circuit = convert_circuit + dis_circuit
        self.quantum_operation = tfq.layers.ControlledPQC(circuit, ops, differentiator=tfq.differentiators.Adj
        self.quantum_weights = tf.Variable(initial_value=np.random.uniform(0, 2 * np.pi, len(ent_params)), dty
        self.circuit_tensor = tfq.convert_to_tensor([cirq.Circuit()])

    def call(self, inputs):
        circuit_batch_dim = tf.gather(tf.shape(inputs), 0)
        tiled_b = tf.tile(tf.expand_dims(self.quantum_weights, 0), [circuit_batch_dim, 1])
        quantum_inputs = tf.concat([inputs, tiled_b], axis=1)
        tiled_circuits = tf.tile(self.circuit_tensor, [circuit_batch_dim])
        quantum_output = self.quantum_operation([tiled_circuits, quantum_inputs])
        return quantum_output

generator = tf.keras.models.Sequential([
  tf.keras.layers.Flatten(input_shape=(4, 4)),
  tf.keras.layers.Dense(128, activation='relu'),
  tf.keras.layers.Dropout(0.2),
  tf.keras.layers.Dense(16)
])

discriminator = tf.keras.Sequential([
    tf.keras.layers.Input(shape=(16,)),
    Hybrid_Dis(),
])

dis_opt = tf.keras.optimizers.Adam()
gen_opt = tf.keras.optimizers.Adam()

@tf.function
def grad(noise, real_input):
  with tf.GradientTape() as gen_tape, tf.GradientTape() as disc_tape:
    generated_images = generator(noise, training=True)

    real_output = discriminator(real_input, training=True)
    fake_output = discriminator(generated_images, training=True)

    gen_loss = generator_loss(fake_output)
    disc_loss = discriminator_loss(real_output, fake_output)

  grads = gen_tape.gradient(gen_loss, generator.trainable_variables)
  gen_opt.apply_gradients(zip(grads, generator.trainable_variables))
  grads = disc_tape.gradient(disc_loss, discriminator.trainable_variables)
  dis_opt.apply_gradients(zip(grads, discriminator.trainable_variables))

noise = tf.random.uniform(shape=[10, 4, 4])
real_input = tf.random.uniform(shape=[10, 4, 4])
real_input = tf.reshape(real_input, [10, 16])
grad(noise, real_input)
```

```
grad(noise, real_input)
```

Shuhul24 on Aug 24, 2022 — Author

Thank you so much ! Finally it worked for me.

Lastly, I want to ask something. Is there a reference/paper/research blog where it's mentioned about an upper bound on number of qubits that needs to be considered for calculation/measurement?

Shuhul24 closed this as completed on Aug 24, 2022

lockwo on Aug 24, 2022 — Contributor

I don't quite understand what you mean by "qubits needed for measurement", could you clarify or give an example? Stuff like VQE has a lot of information because they are constrained by their hamiltonians, but for stuff like a QGAN (or QML systems) you can often just measure 1 qubit.

lockwo mentioned this on Aug 24, 2022

TypeError: Invalid input argument for exponent. (tfq.convert_to_tensor) #712

Shuhul24 on Aug 24, 2022 — Author

I meant like in the above example, we are considering 16 qubits which I don't think is practically possible for gate-based quantum computer to handle. So is there somewhat an upper bound as to what should be maximum number of qubits ? Hope I am clear.

lockwo on Aug 24, 2022 — Contributor

I see now. That's not a question with a simple answer, there are a ton of factors that go into running stuff on real hardware. 16 qubits is definitely possible, but it depends a lot on depth and number of 2 qubit gates. 16 qubit with depth 1 and no 2 qubit gates will be easy, 16 qubits with 200 qubit gates will not be. There is a decent amount of literature on the limits of hardware (e.g. https://arxiv.org/abs/2202.11045) that might be of interest to you.